MILITARY ASSET MANAGEMENT SYSTEM

Enterprise Asset Tracking and Management Application

Project Documentation Report

Developed by: Bhavya

Full-Stack Web Application

1. Project Overview

The Military Asset Management System is a full-stack web application designed to manage and track military assets across multiple bases. The application provides functionality for managing assets, purchases, cross-base transfers, personnel assignments, expenditures, dashboard metrics, authentication, role-based access control, and audit records.

The system uses a React-based frontend and a Node.js/Express backend connected to a PostgreSQL relational database. JWT-based authentication is used to protect authenticated API operations, while role-based authorization controls access to protected operations.

2. Project Objectives

• Provide centralized visibility of military assets across multiple bases.

• Maintain asset quantities by base and equipment type.

• Record purchases and incoming stock.

• Track transfers between military bases.

• Record personnel assignments.

• Record asset expenditures.

• Provide dashboard information for inventory movements.

• Protect APIs using JWT authentication.

• Enforce role-based access control.

• Maintain audit records for important asset-changing operations.

• Use PostgreSQL relational constraints to maintain data integrity.

3. Technology Stack

Layer	Technology
Frontend	React, Vite, CSS
HTTP Client	Axios
Backend	Node.js, Express.js
Authentication	JSON Web Token (JWT)
Authorization	Role-Based Access Control (RBAC)
Security	Helmet, CORS
Database	PostgreSQL
Database Hosting	Neon PostgreSQL
Backend Hosting	Render
Frontend Hosting	Vercel
Version Control	Git and GitHub
API Testing	Postman / API testing tools

4. System Architecture

The application follows a client-server architecture. The React frontend communicates with the Express backend through REST API endpoints. The backend handles authentication, authorization, business operations, database queries, and audit logging.

Layer	Responsibility
React Frontend	User interface, forms, dashboard and navigation
Axios API Client	Communicates with backend REST APIs and sends JWT
Express Backend	Routing, controllers, authentication and business logic
JWT Middleware	Validates authentication tokens
RBAC Middleware	Checks permitted user roles
PostgreSQL	Stores users, bases, assets and transaction records
Neon	Cloud PostgreSQL database hosting
Render	Backend API deployment
Vercel	Frontend deployment

5. User Roles and Role-Based Access Control

The system supports three user roles: ADMIN, BASE_COMMANDER and LOGISTICS_OFFICER. User roles are stored in the users table and are included in the authenticated JWT information.

Role	Purpose
ADMIN	Global administrative access to system operations.
BASE_COMMANDER	Base-level management operations such as assets, assignments and expenditures.
LOGISTICS_OFFICER	Operational management of purchases and transfers.

Authentication Flow

When a protected endpoint is requested, the authentication middleware reads the Authorization header and expects a Bearer token. The JWT is verified using the configured JWT secret. The decoded user information is then attached to req.user.

Authorization Flow

The authorizeRoles middleware receives the allowed roles for an endpoint and checks whether the authenticated user's role is included in the allowed role list. Unauthorized users receive an HTTP 403 response.

6. Database Design

PostgreSQL is used as the relational database. The schema contains nine primary tables that represent bases, users, equipment, assets, purchases, transfers, assignments, expenditures and audit logs.

Table	Purpose
bases	Stores military base information.
users	Stores usernames, password hashes, roles and base relationships.
equipment_types	Stores equipment names and categories.
assets	Stores current asset quantities by base and equipment type.

purchases	Records purchased or incoming assets.
transfers	Records movement of assets between bases.
assignments	Records allocation of assets to personnel.
expenditures	Records consumed or expended assets.
audit_logs	Stores audit information for system actions.

Database Integrity

• Primary keys uniquely identify records.

• Foreign keys maintain relationships between tables.

• CHECK constraints restrict invalid roles, categories and quantities.

• Unique constraint prevents duplicate base/equipment asset combinations.

• Indexes are created on frequently queried fields such as base IDs and equipment type IDs.

• Transfer records prevent transfers from a base to itself.

7. Core Features

Authentication

Users can register and log in through the authentication API. Passwords are securely hashed and authentication is handled using JWT tokens.

Asset Management

The asset module manages asset quantities associated with bases and equipment types. Authorized users can create and update asset records.

Purchases

The purchases module records incoming assets, including base, equipment type, quantity and the user responsible for creating the record.

Transfers

The transfer module records movement of equipment between source and destination bases. Each transfer records the equipment type, quantity, status and initiating user.

Assignments

The assignment module records equipment allocated to personnel at a base.

Expenditures

The expenditure module records assets consumed or expended, including quantity, reason, base and responsible user.

Dashboard

The dashboard provides summarized inventory and movement information for authenticated users.

8. Inventory Calculation

The application uses inventory movement information to provide a view of asset quantities and movements. The core business concept is based on opening balance, movements, assignments, expenditures and closing balance.

Closing Balance = Opening Balance + Net Movement - Assigned - Expended

Net Movement = Purchases + Transfers In - Transfers Out

This model allows the dashboard to present meaningful inventory information without requiring separate manually maintained totals.

9. API Endpoints

Module	Endpoint	Method
Authentication	/api/auth/register	POST
Authentication	/api/auth/login	POST
Authentication	/api/auth/me	GET
Authentication	/api/auth/admin-test	GET
Assets	/api/assets	GET
Assets	/api/assets	POST
Assets	/api/assets/:id	PUT
Purchases	/api/purchases	GET
Purchases	/api/purchases	POST
Transfers	/api/transfers	GET
Transfers	/api/transfers	POST
Assignments	/api/assignments	GET
Assignments	/api/assignments	POST
Expenditures	/api/expenditures	GET
Expenditures	/api/expenditures	POST
Dashboard	/api/dashboard/summary	GET
Health	/api/health	GET

10. Authentication and Security

• JWT tokens are used for authentication.

• Protected endpoints require a valid Bearer token.

• Role-based authorization is applied to restricted operations.

• Passwords are stored as hashes rather than plain text.

• Helmet is enabled to provide security-related HTTP headers.

• CORS is configured on the backend.

• PostgreSQL constraints help prevent invalid database records.

• Environment variables are used for sensitive configuration such as database credentials and JWT secrets.

11. Audit Logging

The audit_logs table provides a centralized record of system actions. Each record can store the responsible user, action type, descriptive details and timestamp.

Field	Description
id	Unique audit record identifier.
user_id	User responsible for the action.
action	Action type such as PURCHASE or TRANSFER.
details	Description of the operation.
created_at	Timestamp of the audit record.

12. Frontend Screens

• Login screen for user authentication.

• Dashboard for inventory and movement summaries.

• Inventory screen for asset information.

• Purchases screen for recording and viewing purchases.

• Transfers screen for recording and viewing transfers.

• Assignments screen for personnel asset assignments.

• Expenditures screen for recording consumed assets.

• Navigation and API integration through Axios.

13. Deployment

The backend API is deployed on Render and the frontend application is deployed on Vercel. PostgreSQL is hosted using Neon.

Backend

The backend is deployed as a Node.js web service. The production server uses the PORT environment variable supplied by the hosting platform.

Frontend

The React/Vite frontend is deployed on Vercel as a production web application.

Database

The PostgreSQL database is hosted on Neon and accessed by the backend using the configured database connection.

14. Setup Instructions

Backend:

• Navigate to the backend directory.

• Install dependencies using npm install.

• Configure database and JWT environment variables.

• Start the development server using npm run dev.

• For production, use npm start.

Frontend:

• Navigate to the frontend directory.

• Install dependencies using npm install.

• Configure the backend API base URL.

• Start the development server using npm run dev.

15. Sample Test Credentials

Role	Username	Password	Base
Admin	admin_user	AdminPass123!	All Bases
Base Commander	commander_alpha	CommandPass123!	Fort Alpha / Base #1
Logistics Officer	logistics_officer	LogisticsPass123!	Base #1 / Global Ops

Note: Test credentials should be used only for evaluation/demo purposes and should not be reused for production systems.

16. Live Application Links

Frontend:
https://military-asset-management-sable.vercel.app/

Backend API:
https://military-asset-management-backend-44bl.onrender.com

GitHub Repository:
https://github.com/bhavya-Yerramanayuni/military-asset-management

17. Conclusion

The Military Asset Management System provides a structured full-stack solution for managing military assets across multiple bases. The system combines React, Node.js, Express and PostgreSQL to provide asset tracking, movement management, assignments, expenditures, authentication, role-based authorization and auditability.

The application has been version-controlled using Git and GitHub and deployed using Vercel for the frontend, Render for the backend and Neon for PostgreSQL database hosting.

The resulting system demonstrates the implementation of a modular full-stack application with relational data integrity, secured APIs and operational asset management functionality.